Estratégia Concursos

Aula 33

Polícia Federal (Agente de Polícia) Passo Estratégico de Informática - Prof Diego Carvalho

Autor:
Diego Carvalho

09 de Abril de 2025

98276239072 - FABIANO CARBONELL DA SILVA

Índice

1) O que é mais cobrado no assunto - Processos de Desenvolvimento - CEBRASPE 3

2) Roteiro de Revisão - Processos de Desenvolvimento 4

3) Aposta Estratégica - Processos de Desenvolvimento 17

4) Questões Estratégicas - Processos de Desenvolvimento - CEBRASPE 19

5) Questionário de Revisão - Processos de Desenvolvimento 25

6) Lista de Questões Estratégicas - Processos de Desenvolvimento - CEBRASPE 34

7) Referências Bibliográficas - Processos de Desenvolvimento 36

8) Gabarito de Questões Estratégicas - Processos de Desenvolvimento - CEBRASPE 37

98276239072 - FABIANO CARBONELL DA SILVA

O QUE É MAIS COBRADO DENTRO DO ASSUNTO?

Considerando os tópicos que compõem o nosso assunto, possuímos a seguinte distribuição percentual:

TÓPICO	% DE COBRANÇA [CEBRASPE]
Conceitos Básicos	16%
Modelo em Cascata	42%
RAD	03%
Modelos Evolucionários	01%
Modelos em Prototipagem	14%
Modelo em Espiral	22%
Modelo Baseado em Componentes	02%

98276239072 - FABIANO CARBONELL DA SILVA

Roteiro de Revisão e Pontos do Assunto que Merecem Destaque

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

ENGENHARIA DE SOFTWARE

DEFINIÇÕES DE CICLO DE VIDA

Ciclo De Vida

- Trata-se das fases pelas quais alguma coisa passa desde o seu início até o seu fim -

Ciclo De Vida de software

- Trata-se das fases pelas quais um software passa desde o seu início até o seu fim -

Modelo de Ciclo De Vida de Software

- Trata-se das fases pelas quais um software passa desde o seu início até o seu fim e como essas fases se relacionam -

98276239072 - FABIANO CARBONELL DA SILVA

CICLOS DE VIDA GENÉRICOS

CICLOS DE VIDA GENÉRICOS

Processos de Software

- Conjunto de atividades, métodos, práticas e transformações que guiam pessoas na produção de software -

Modelo de Processo de software

- Mesmo conceito de Modelo de Ciclo de Vida - é uma representação abstrata de um processo de software -

DESCRIÇÃO DE FASES GENÉRICAS

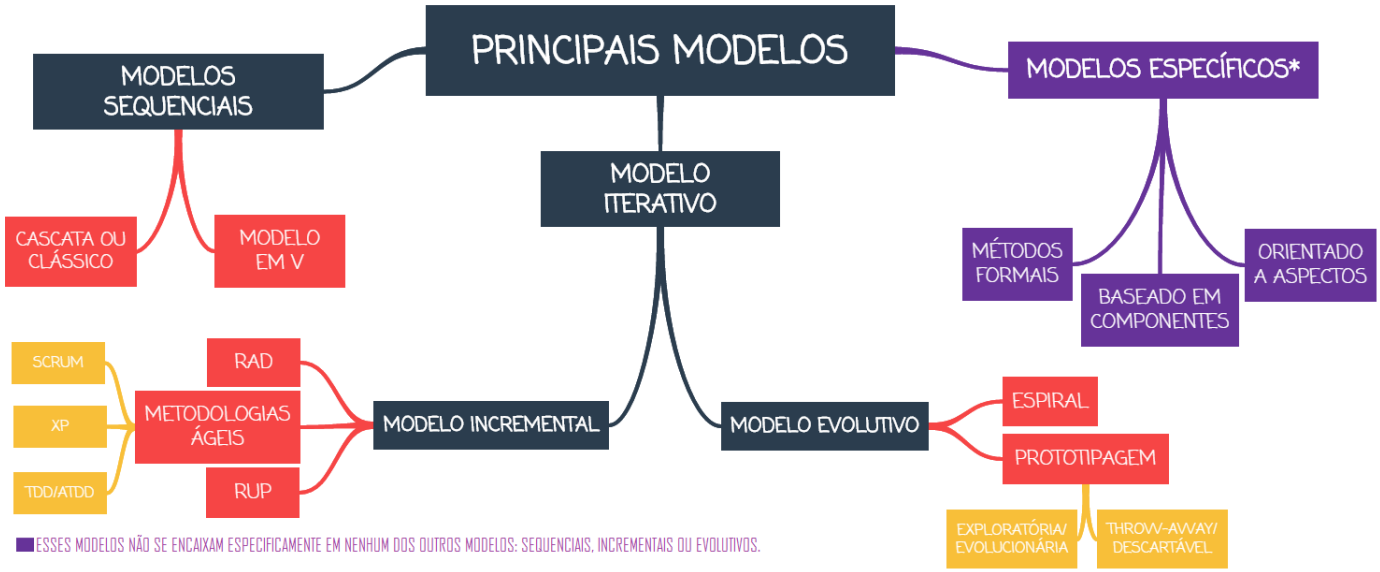
98276239072 - FABIANO CARBONELL DA SILVA

FASES	DESCRIÇÃO
PLANEJAMENTO	O objetivo do planejamento de projeto é fornecer uma estrutura que possibilite ao gerente fazer estimativas razoáveis de recursos, custos e prazos. Uma vez estabelecido o escopo de software, com um pequeno esboço dos requisitos, uma proposta de desenvolvimento deve ser elaborada, isto é, um plano de projeto deve ser elaborado configurando o processo a ser utilizado no desenvolvimento de software. À medida que o projeto progride, o planejamento deve ser detalhado e atualizado regularmente. Pelo menos ao final de cada uma das fases do desenvolvimento (análise e especificação de requisitos, projeto, implementação e testes), o planejamento como um todo deve ser revisto e o planejamento da etapa seguinte deve ser detalhado. O planejamento e o acompanhamento do progresso fazem parte do processo de gerência de projeto.
ANÁLISE E ESPECIFICAÇÃO DE REQUISITOS	Nesta fase, o processo de levantamento de requisitos é intensificado. O escopo deve ser refinado e os requisitos mais bem definidos. Para entender a natureza do software a ser construído, o engenheiro de software tem de compreender o domínio do problema, as restrições, as metas, as funcionalidades e o comportamento esperados – ele pode o fazer por meio de diversas técnicas de levantamento de requisitos (Ex: entrevistas). Uma vez capturados os requisitos do sistema a ser desenvolvido, estes devem ser modelados, avaliados e documentados. Uma parte vital desta fase é a construção de um modelo descrevendo o que o software tem de fazer (e não como fazê-lo).
PROJETO	Esta fase é responsável por incorporar requisitos tecnológicos aos requisitos essenciais do sistema, modelados na fase anterior e, portanto, requer que a plataforma de implementação seja conhecida. Basicamente, envolve duas grandes etapas: projeto da arquitetura do sistema e projeto detalhado. O objetivo da primeira etapa é definir a arquitetura geral do software, tendo por base o modelo construído na fase de análise de requisitos. Essa arquitetura deve descrever a estrutura de nível mais alto da aplicação e identificar seus principais componentes. O propósito do projeto detalhado é detalhar o projeto do software para cada componente identificado na etapa anterior. Os componentes de software devem ser sucessivamente refinados em níveis maiores de detalhamento (inclusive em relação à tecnologia adotada) até que possam ser codificados e testados.
IMPLEMENTAÇÃO	O projeto deve ser traduzido para uma forma passível de execução pela máquina. A fase de implementação realiza esta tarefa, isto é, cada unidade de software do projeto detalhado é implementada.
TESTES	Inclui diversos níveis de testes, a saber, teste de unidade, teste de integração e teste de sistema. Inicialmente, cada unidade de software implementada deve ser testada e os resultados documentados. A seguir, os diversos componentes devem ser integrados sucessivamente até se obter o sistema. Finalmente, o sistema como um todo deve ser testado.
ENTREGA E IMPLANTAÇÃO	Uma vez testado, o software deve ser colocado em produção. Para tal, contudo, é necessário treinar os usuários, configurar o ambiente de produção e, muitas vezes, converter bases de dados. O propósito desta fase é estabelecer que o software satisfaz os requisitos dos usuários. Isto é feito instalando o software e conduzindo testes de aceitação. Quando o software tiver demonstrado prover as capacidades requeridas, ele pode ser aceito e a operação iniciada.

98276239072 - FABIANO CARBONELL DA SILVA

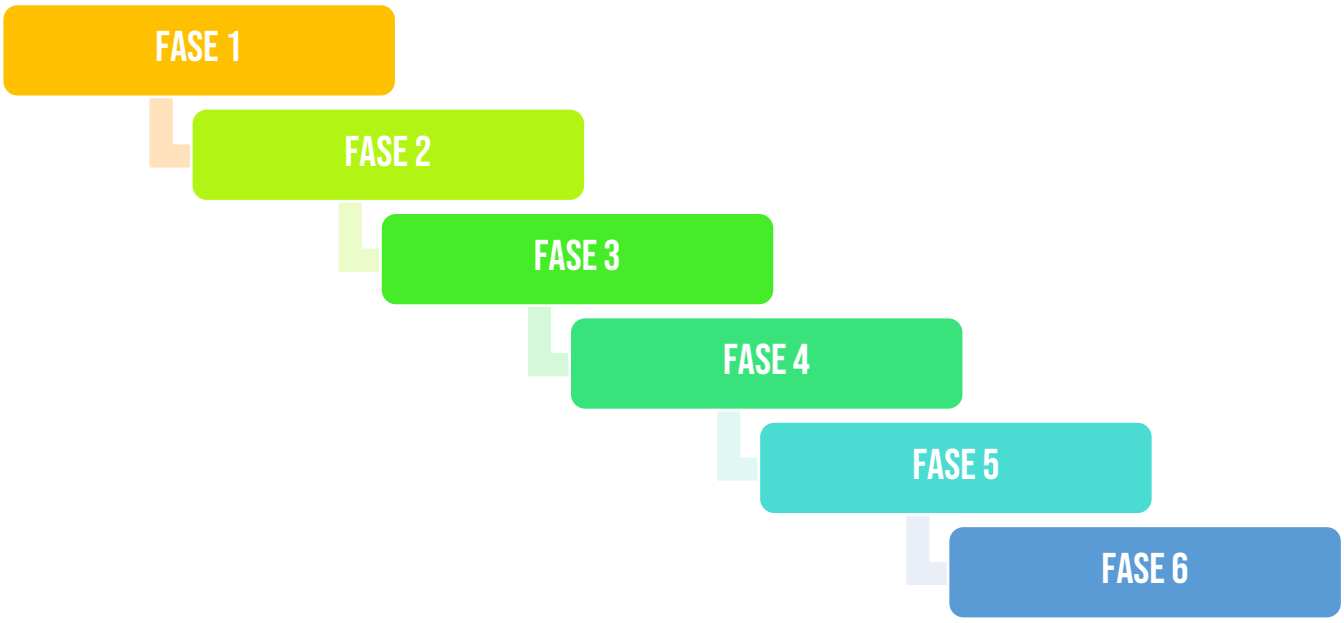
OPERAÇÃO	Nesta fase, o software é utilizado pelos usuários no ambiente de produção, isto é, no ambiente real de uso do usuário.
MANUTENÇÃO	Indubitavelmente, o software sofrerá mudanças após ter sido entregue para o usuário. Alterações ocorrerão porque erros foram encontrados, porque o software precisa ser adaptado para acomodar mudanças em seu ambiente externo, ou porque o cliente necessita de funcionalidade adicional ou aumento de desempenho. Muitas vezes, dependendo do tipo e porte da manutenção necessária, essa fase pode requerer a definição de um novo processo, onde cada uma das fases precedentes é reaplicada no contexto de um software existente ao invés de um novo.

PRINCIPAIS MODELOS DE DESENVOLVIMENTO



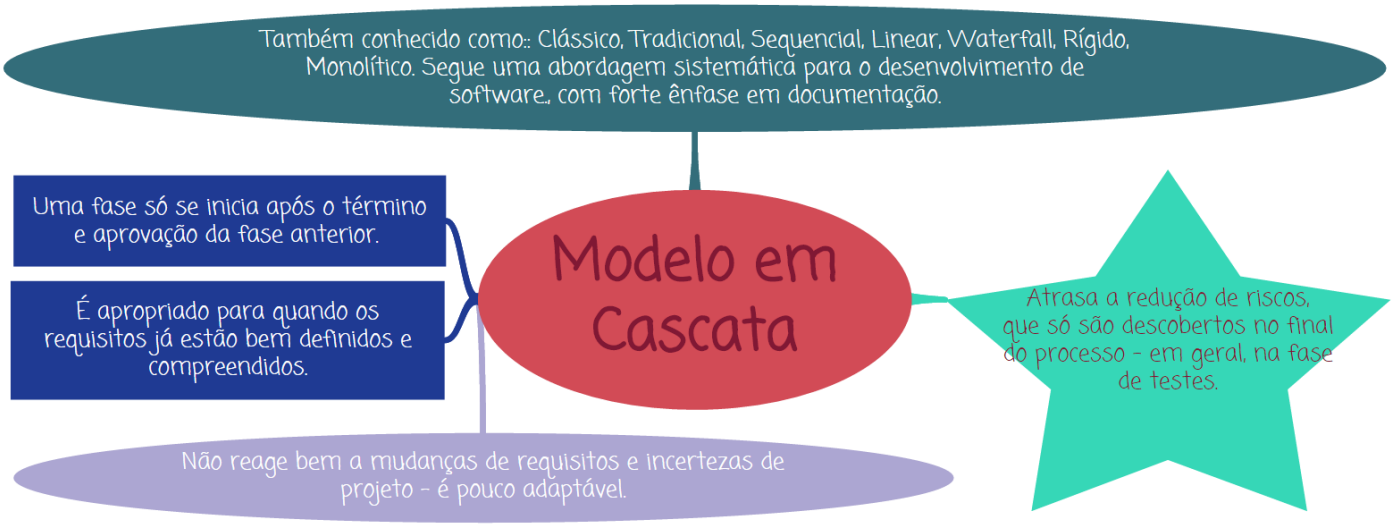
MODELOS EM CASCATA

98276239072 - FABIANO CARBONELL DA SILVA



POR SOMMERVILLE	POR ROYCE	POR PRESSMAN (4ª ED)	POR PRESSMAN (6ª ED)
Análise e Definição de Requisitos	Requisitos de Sistema	Modelagem e Engenharia do Sistema/Informação	Comunicação
Projeto de Sistema e Software	Requisitos de Software	Análise de Requisitos de Software	Planejamento
Implementação e Teste de Unidade	Análise	Projeto	Modelagem
Integração e Teste de Sistema	Projeto	Geração de Código	Construção
Operação e Manutenção	Codificação	Teste e Manutenção	Implantação
	Teste		
	Operação		

98276239072 - FABIANO CARBONELL DA SILVA



VANTAGENS	DESVANTAGENS
É simples de entender e fácil de aplicar, facilitando o planejamento.	Divisão inflexível do projeto em estágios distintos.
Fixa pontos específicos para a entrega de artefatos.	Dificuldade em incorporar mudanças de requisitos.
Funciona bem para equipes tecnicamente fracas.	Clientes só visualizam resultados próximos ao final do projeto.
É fácil de gerenciar, devido a sua rigidez.	Atrasa a redução de riscos.
Realiza documentação extensa por cada fase ou estágio.	Apenas a fase final produz um artefato de software entregável.
Possibilita boa aderência a outros modelos de processo.	Cliente deve saber todos os requisitos no início do projeto.
Funciona bem com projetos pequenos e com requisitos bem conhecidos.	Modelo inicial (Royce) não permitia feedback entre as fases do projeto.
	Pressupõe que os requisitos ficarão estáveis ao longo do tempo.
	Não funciona bem com projetos complexos e OO, apesar de compatível.

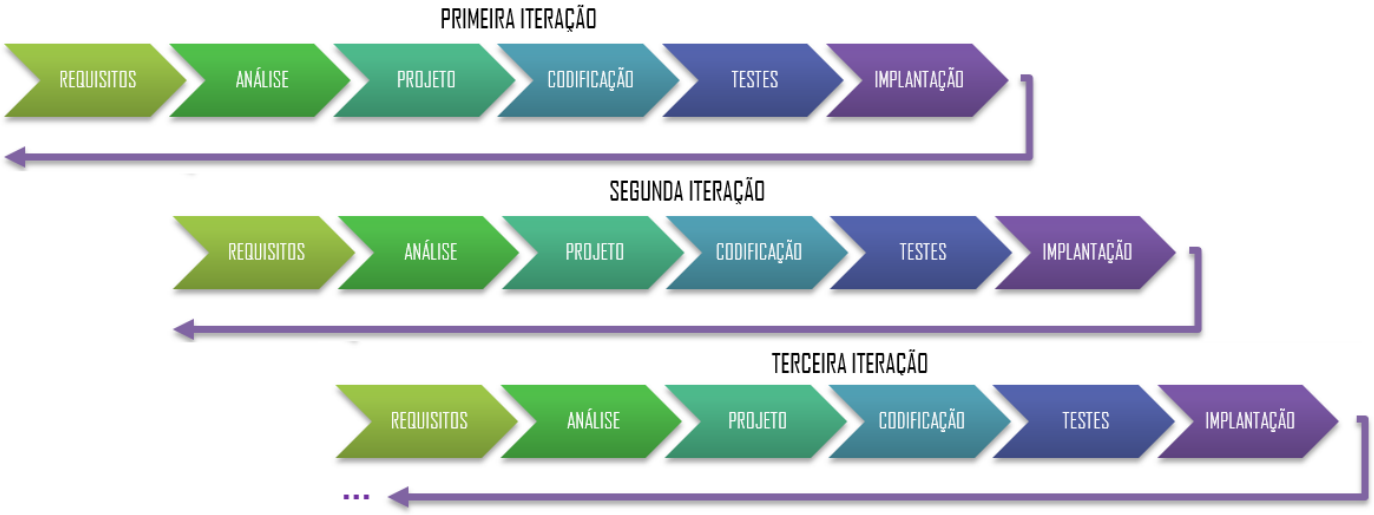
MODELO ITERATIVO E INCREMENTAL

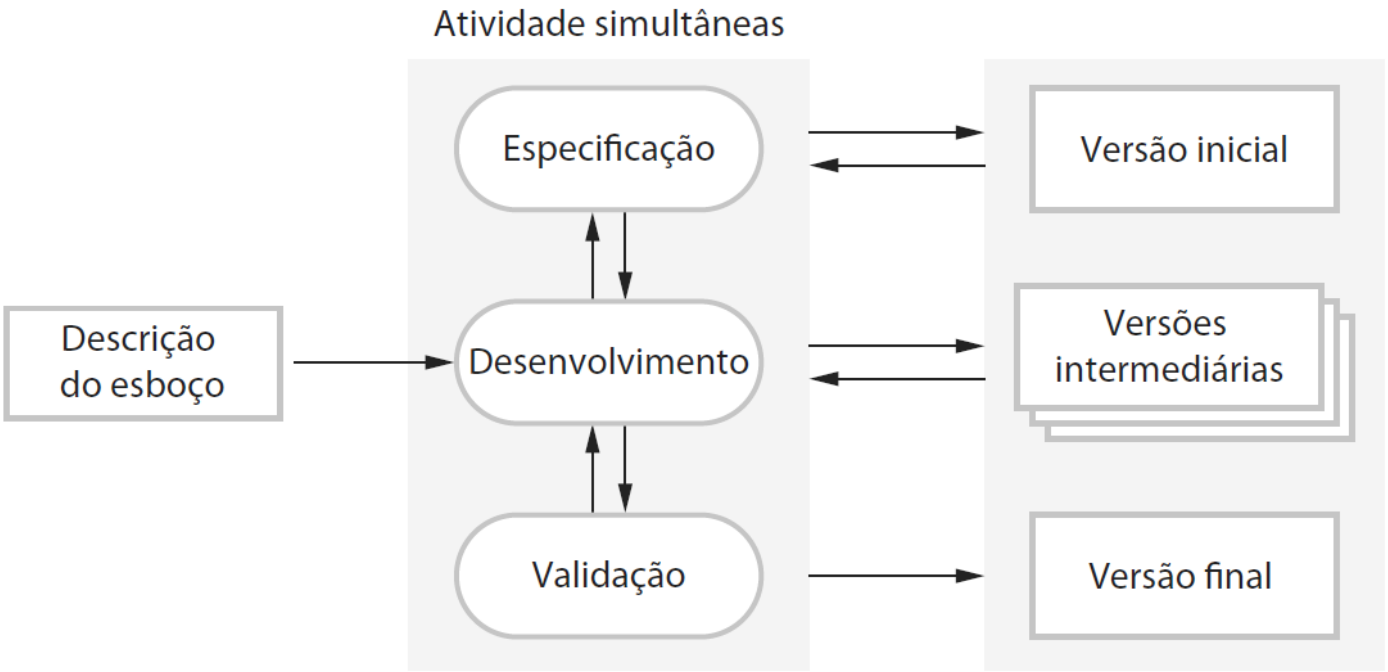
Modelo Incremental: observem que a imagem mostra um artista com uma ideia completa já sobre o quadro em sua cabeça, mas ele desenvolve cada parte do quadro separadamente até integrar as partes em uma imagem completa. É como se fosse um quebra-cabeças em que cada

98276239072 - FABIANO CARBONELL DA SILVA

parte é entregue funcionando e depois integrada. **Ele produz *builds*, isto é, partes do software**.

Modelo Iterativo: observem que a imagem mostra um artista com um esboço do quadro, sendo que ele desenvolve várias versões até chegar ao resultado final que ele deseja. É como se fosse uma visão abstrata da imagem, que em seguida vai sendo melhorada até chegar a uma visão mais concreta. **Ele produz *releases*, isto é, versões constantemente melhoradas do software**.





98276239072 - FABIANO CARBONELL DA SILVA

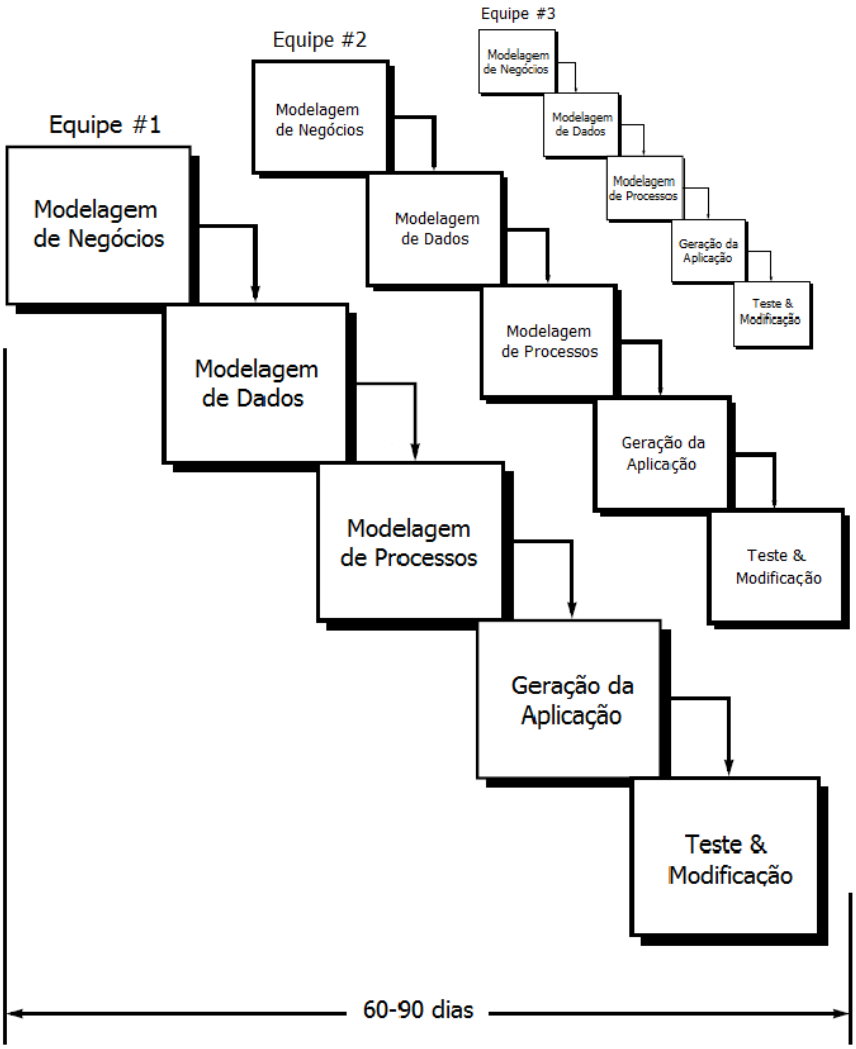
RAPID APPLICATION DEVELOPMENT (RAD)

FASES	DESCRIÇÃO
MODELAGEM DE NEGÓCIO	O fluxo de informações entre as funções de negócio é modelado de modo a responder que informação direciona o processo de negócio; que informação é gerada; quem gera essa informação; para onde vai a informação gerada; e, por fim, quem processa a informação.
MODELAGEM DE DADOS	O fluxo de informação definido na fase de modelagem de negócio é refinado em um conjunto de objetos de dados que são necessários para suportar o negócio. Os atributos de cada objeto são identificados e os relacionamentos entre esses objetos são definidos.
MODELAGEM DE PROCESSO	Os objetos de dados definidos na modelagem de dados são transformados para conseguir o fluxo necessário para implementar uma função do negócio. Descrições do processamento são criadas para adicionar, modificar, descartar ou recuperar um objeto de dados.
GERAÇÃO DA APLICAÇÃO	Considera o uso de técnicas de quarta geração, trabalha com a reutilização de componentes de programa existentes quando possível, ou cria componentes reusáveis. São usadas ferramentas automatizadas para facilitar a construção do software.
TESTE E MODIFICAÇÃO	Como o processo enfatiza o reúso, muitos componentes já estão testados e isso reduz o tempo total de teste. No entanto, os novos componentes devem ser testados e todas as interfaces devem ser exaustivamente exercitadas para colocar o resultado em produção.

VANTAGENS	DESVANTAGENS
Permite o desenvolvimento rápido e/ou a prototipagem de aplicações.	Exige recursos humanos caros e experientes.
Criação e reutilização de componentes.	O envolvimento com o usuário tem que ser ativo.
Desenvolvimento é conduzido em um nível mais alto de abstração.	Comprometimento da equipe do projeto.
Grande redução de codificação manual com *wizards*.	Custo alto do conjunto de ferramentas e hardware para rodar a aplicação;
Cada função pode ser direcionada para a uma equipe separada.	Mais difícil de acompanhar o projeto.
Maior flexibilidade (desenvolvedores podem reprojetar à vontade).	Perda de precisão científica (pela falta de métodos formais).
Provável custo reduzido (tempo é dinheiro e também devido ao reuso).	Pode levar ao retorno das práticas caóticas no desenvolvimento.
Tempo de desenvolvimento curto.	Pode construir funções desnecessárias.
Protótipos permitem uma visualização mais cedo.	Requisitos podem não se encaixar (conflitos entre desenvolvedores e clientes).

98276239072 - FABIANO CARBONELL DA SILVA

Envolvimento maior do usuário.	Padronização (aparência diferente entre os módulos e componentes)

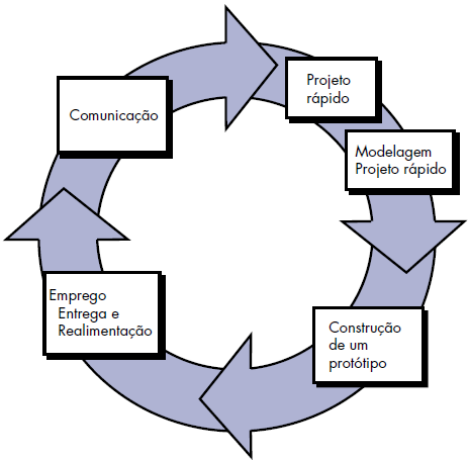


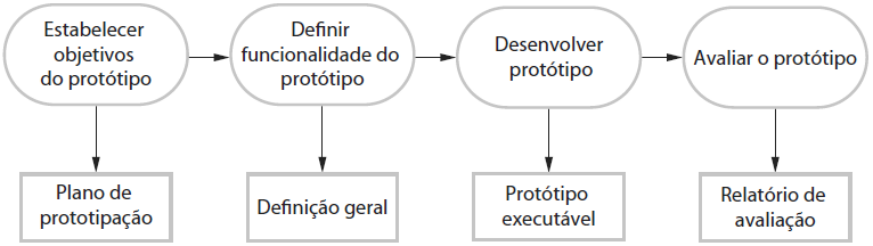
MODELO EVOLUCIONÁRIO

Modelo Evolucionário x Iterativo: o modelo evolucionário funciona também com base em iterações ou repetições com o intuito de refinar o software a partir do feedback do cliente. A ideia é utilizar um esboço inicial e ir incrementando, melhorando, aperfeiçoando o software de acordo com as necessidades dos clientes até chegar a uma versão. Ao fim, o sistema pode ser entregue ao cliente ou ser refeito de forma mais estruturada.

MODELO EM PROTOTIPAGEM

98276239072 - FABIANO CARBONELL DA SILVA





DESENVOLVIMENTO EXPLORATÓRIO OU EVOLUCIONÁRIO	O objetivo do processo de desenvolvimento exploratório é trabalhar com o cliente para explorar os requisitos e entregar um sistema final. O desenvolvimento começa com as partes do sistema compreendidas. O sistema evolui por meio da adição de novas características propostas pelo cliente.
PROTOTIPAÇÃO THROWAWAY OU DESTARTÁVEL	O objetivo do processo de prototipação throwaway é compreender os requisitos do cliente e, a partir disso, desenvolver melhor definição de requisitos para o sistema. O protótipo se concentra na experimentação dos requisitos mal compreendidos do cliente, mas é posteriormente descartado.

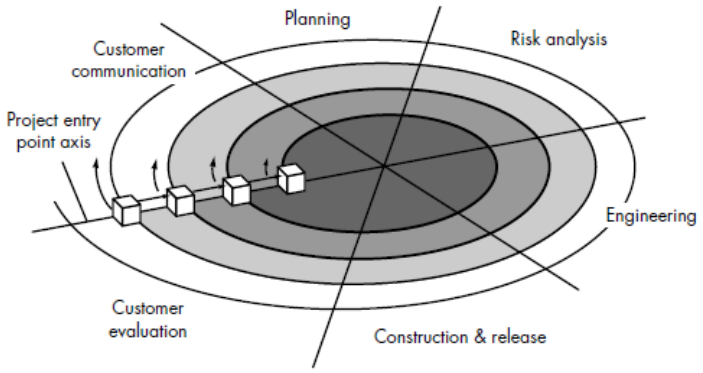
MODELO EM ESPIRAL

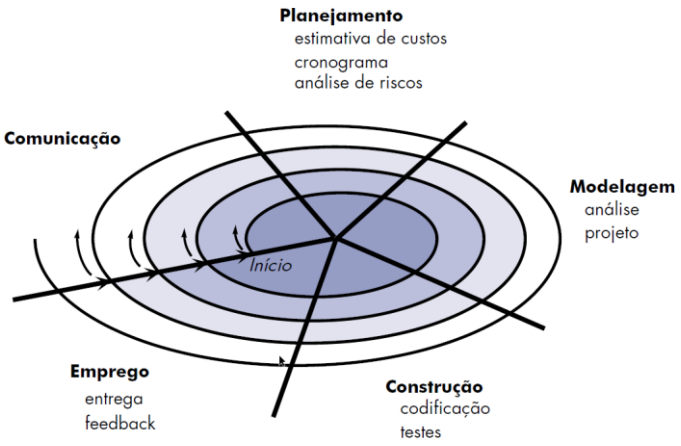
Q Modelo em Espiral foi proposto originalmente, em 1988, por Boehm. Sua ideia era representar um processo de software orientado a riscos. Em vez de representar o processo como uma sequência de atividades com algum retorno entre uma atividade e outra, o processo é representado como uma espiral. É conhecido como prototipagem-em-etapas, por combinar, em geral, o modelo em cascata com a prototipação. Cada loop representa uma fase do processo de software.

SETORES (POR BOEHM)	DESCRIÇÃO
DETERMINAR OBJETIVOS, ALTERNATIVAS E RESTRIÇÕES	Definem-se os objetivos da do projeto (aumentar performance, consertar funcionalidade, melhorar qualidade), identificam-se alternativas (reúso de componentes, comprar pronto) e identificam-se restrições impostas (custo, cronograma, entre outros).
AVALIAR ALTERNATIVAS, IDENTIFICAR E RESOLVER RISCOS	Avaliam-se as alternativas identificadas em relação aos objetivos e restrições. Frequentemente, esse processo identifica áreas de incerteza (custos excessivos, falta de recursos) e resolve ou reduz os riscos identificados - um protótipo pode ser construído.
DESENVOLVER E TESTAR	Após a redução dos riscos, um modelo de desenvolvimento para o sistema é selecionado (Exemplo: prototipação evolucionária, modelos formais, modelo em cascata, modelos baseados em componentes). O software é desenvolvido e, posteriormente, testado.
PLANEJAR PRÓXIMAS FASES	Revisa-se o projeto e decide-se sobre o prosseguimento ao próximo loop da espiral. Se a decisão for pelo prosseguimento, são

98276239072 - FABIANO CARBONELL DA SILVA

	elaborados planos para a próxima fase do projeto, e terminamos um loop.





VANTAGENS	DESVANTAGENS
Suporta mecanismos de redução de riscos.	Exige analistas de risco bastante experientes.
Obtêm-se versões do sistema a cada iteração.	Exige uma equipe de desenvolvimento extremamente qualificada.
Entregando produtos cada vez mais refinados e de melhor qualidade.	Exige um gerenciamento de processo mais complexo.
Reflete as práticas reais de engenharia atual.	Não é recomendado resolver problemas mais simples e pequenos.
Apresenta uma abordagem sistemática.	
Apresenta estimativas realistas.	

MÉTODOS FORMAIS

Métodos Formais permitem indicar atividades que contem com representações matemáticas de software, especificação formal, prova de especificação, desenvolvimento transformacional, etc. Esse modelo é utilizado em ambientes extremamente complexos. São bastante lentos e dispendiosos, além de exigirem um treinamento intensivo. Em geral, são utilizados para o desenvolvimento de sistemas que necessitam de grande robustez e confiabilidade diante da possibilidade de perda de vidas ou sério prejuízo, caso haja falhas.

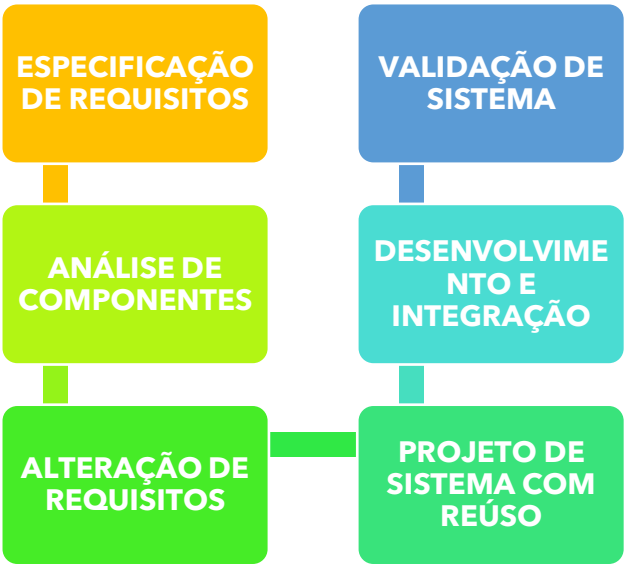
ETAPAS	DESCRIÇÃO
ESPECIFICAÇÃO FORMAL PARA DEFINIÇÃO DE REQUISITOS	Busca formalizar os requisitos descobertos utilizando algum método formal, criando uma modelagem com o uso de elementos.
REFINAMENTO PARA CONCEPÇÃO DE PROJETO	Busca conceber o projeto de software, ou seja, nesta fase é pensado na arquitetura do sistema com seus diversos componentes e suas interfaces, dados e relacionamentos entre os mesmos.

98276239072 - FABIANO CARBONELL DA SILVA

SÍNTESE PARA IMPLEMENTAÇÃO	Busca gerar esqueleto de código, a partir do modelo refinado, que pode servir como base para implementação real do sistema.
PROTOTIPAÇÃO PARA VALIDAÇÃO	Busca elaborar um protótipo funcional do sistema para validar se o mesmo, atende as necessidades do cliente.
PROVA PARA A VERIFICAÇÃO	Busca verificar se o sistema desenvolvido foi concebido atendendo a todos os Requisitos Funcionais e Não-Funcionais elicitados.

MODELO BASEADO EM COMPONENTES

O **Modelo Baseado em Componentes** tem utilizado o reúso como peça principal. Essa abordagem depende de uma grande base de componentes reusáveis e algum framework de integração.



FASES DO MODELO BASEADO EM COMPONENTES	DESCRIÇÃO
ESPECIFICAÇÃO DE REQUISITOS	Tem o objetivo de traduzir as informações coletadas durante a atividade de análise em um documento que define um conjunto de requisitos de software. Devem ser incluídos dois tipos de requisitos nesse documento: os Requisitos de Usuário e Requisitos de Sistema.
ANÁLISE DE COMPONENTES	Nesta fase, é feita uma busca pelos componentes para implementar essa especificação. Geralmente, não existe uma correspondência exata entre o componente encontrado e o procurado. Muitas vezes, os componentes que podem ser usados fornecem apenas parte da funcionalidade necessária.
MODIFICAÇÃO DE REQUISITOS	Os requisitos são analisados usando as informações sobre os componentes encontrados, que são modificados para refletir os componentes disponíveis. Quando as modificações são impossíveis, a atividade de análise de componentes pode ser novamente realizada para procurar alternativas.

98276239072 - FABIANO CARBONELL DA SILVA

PROJETO DE SISTEMA COM REÚSO	O framework do sistema é projetado ou um framework existente é reutilizado. Os projetistas levam em consideração os componentes reusados. Pode ser necessário projetar algum software novo caso os componentes reusáveis não estejam disponíveis para aquisição para o sistema.
DESENVOLVIMENTO E INTEGRAÇÃO	Software que não pode ser adquirido externamente é desenvolvido e os componentes e os sistemas COTS são integrados para criar os novos sistemas. A integração de sistema, neste modelo, pode ser parte do processo de desenvolvimento, em vez de ser uma atividade separada.
VALIDAÇÃO DE SISTEMA	Processo de verificação de se um sistema atende às necessidades e expectativas do cliente. *Professor, o que são Sistemas COTS?* Esse é o acrônimo de Commercial Off-The-Shelf, que é um conjunto de soluções pré-fabricadas e disponíveis no mercado, podendo ser compradas ou licenciadas, i.e., uma grande biblioteca de componentes prontos.

MODELO ORIENTADO A ASPECTOS

A **Programação Orientada a Aspectos** é uma abordagem que permite a separação dessas propriedades ortogonais dos componentes funcionais de uma forma natural e concisa, utilizando-se de mecanismos de abstração e de composição para a produção de código executável.

98276239072 - FABIANO CARBONELL DA SILVA

APOSTA ESTRATÉGICA

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais.

Eu listei abaixo os pontos com maior probabilidade de cobrança no contexto de **Processos de Desenvolvimento de Software**. Estas são as minhas apostas:

1. Eu aposto em questões que abordem o **Ciclo de Vida de Software**, que inclui as fases pelas quais um software passa, desde o seu início até o seu fim.

2. Aposto em perguntas sobre as **fases do Ciclo de Vida de Software**, incluindo planejamento, análise e especificação de requisitos, projeto, implementação, testes, entrega e manutenção.

3. Vale a pena revisar o conceito de **modelos de Ciclo de Vida de Software**, que são representações abstratas das fases do desenvolvimento.

4. Acredito que pode aparecer uma questão sobre o **Modelo em Cascata**, que é simples de entender e aplicar, mas com divisões inflexíveis entre as fases do projeto.

5. Uma questão comum seria sobre as **desvantagens do Modelo em Cascata**, como a dificuldade em incorporar mudanças de requisitos ao longo do processo.

6. Eu revisaria o **Modelo Iterativo e Incremental**, que foca em ciclos de desenvolvimento com entregas parciais e versões melhoradas ao longo do tempo.

7. Imagino que poderia haver uma pergunta sobre o **Modelo RAD (Rapid Application Development)**, que permite o desenvolvimento rápido e a criação de protótipos.

8. Aposto em uma questão sobre o **Modelo em Espiral**, que enfatiza a avaliação e mitigação de riscos em cada fase do desenvolvimento.

9. Vale a pena revisar o **conceito de Desenvolvimento Evolutivo**, que trabalha com a evolução do software conforme feedbacks contínuos do cliente.

10. Acredito que pode cair uma questão sobre **Métodos Formais de Desenvolvimento**, que utilizam representações matemáticas para a especificação e verificação de software em ambientes críticos.

11. Apostaria em perguntas sobre o **Modelo Baseado em Componentes**, que utiliza componentes de software reutilizáveis para acelerar o desenvolvimento.

12. Vale revisar as vantagens e desvantagens do **Modelo em Prototipagem**, que permite experimentar com requisitos incertos, mas cujo protótipo é descartado após essa fase.

98276239072 - FABIANO CARBONELL DA SILVA

13. Aposto em perguntas sobre o **Modelo Orientado a Aspectos**, que permite a separação de propriedades ortogonais do código, facilitando o gerenciamento de funcionalidades cruzadas.

14. Pode surgir uma questão sobre a importância da fase de **manutenção do ciclo de vida de software**, que inclui a correção de erros e adaptações às novas necessidades.

15. Revisaria o modelo de **Desenvolvimento Exploratório**, que foca em explorar requisitos com o cliente e evoluir o sistema conforme o feedback, priorizando flexibilidade e adaptação.

98276239072 - FABIANO CARBONELL DA SILVA

QUESTÕES ESTRATÉGICAS

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões.

1. (CESPE / CAU-BR – 2024) Uma vantagem do ciclo de vida do software em cascata é que ele permite modificações nos requisitos por meio de atualização das fases anteriores.

Comentários:

O modelo de ciclo de vida em cascata é caracterizado pela rigidez nas fases, onde cada uma deve ser concluída antes de iniciar a próxima. Mudanças nos requisitos após a fase inicial podem ser muito custosas ou inviáveis, pois requerem revisão das fases já concluídas, o que o modelo não facilita.

Gabarito: Errado

2. (CESPE / CAU-BR – 2024) No modelo em cascata, o ciclo de vida do software torna o processo de desenvolvimento estruturado.

Comentários:

O modelo em cascata é um dos mais antigos paradigmas de desenvolvimento de software, caracterizando-se por uma abordagem sequencial e estruturada, onde cada fase deve ser completada antes de iniciar a próxima. Este modelo é conhecido por sua rigidez e por ser mais adequado a projetos com requisitos bem definidos e imutáveis.

Gabarito: Correto

3. (CESPE / CAU-BR – 2024) Quando se utiliza um modelo incremental, frequentemente o primeiro incremento é um produto essencial; após esse primeiro, se libera uma série de incrementos, que oferecem, progressivamente, maior funcionalidade ao cliente à medida que cada incremento é entregue.

Comentários:

A descrição aborda o modelo incremental em projetos de software, uma técnica de desenvolvimento em que o produto é projetado, implementado e testado incrementalmente até que o produto final esteja completo. Isso permite que versões iniciais do software sejam entregues aos clientes mais cedo e melhoradas através de versões subsequentes, baseadas em feedback e requisitos adicionais.

Gabarito: Correto

98276239072 - FABIANO CARBONELL DA SILVA

4. (CESPE / TCE-SC – 2022) Os modelos de processo de software em cascata e incremental possuem abordagens distintas. O primeiro considera as atividades fundamentais do processo, representando cada uma delas em fases distintas, tais como especificação de requisitos, implementação e teste; o segundo intercala as atividades de especificação, desenvolvimento e validação em uma série de versões do sistema, ao longo do seu ciclo de vida.

Comentários:

O modelo cascata é um processo de desenvolvimento de software linear e sequencial, onde o progresso flui em uma direção, como uma cascata, através das fases de concepção, iniciação, análise, design, construção, teste, implantação e manutenção. O modelo incremental, por outro lado, divide o projeto em incrementos pequenos, permitindo que partes do projeto sejam desenvolvidas em paralelo e integradas conforme completadas, facilitando a entrega de múltiplas versões para revisão do usuário.

Gabarito: Correto

5. (CESPE / PETROBRAS – 2022) No modelo de desenvolvimento Waterfall, a partir da coleta de requisitos e da elaboração do projeto desenvolve-se uma implementação inicial, que é apresentada para a apreciação dos usuários; esse ciclo continua com a criação de várias versões, até que o sistema final seja desenvolvido por meio da execução das etapas de desenvolvimento e testes de forma intercalada.

Comentários:

Na verdade, no Modelo Waterfall (Cascata), as fases são executadas de maneira sequencial e sem retorno a fases anteriores. Não há ciclos de desenvolvimento com várias versões como descrito. Este ciclo de revisão e desenvolvimento de várias versões é mais característico de modelos iterativos e incrementais.

Gabarito: Errado

6. (CESPE / BANRISUL – 2022) Um dos problemas da aplicação do modelo cascata no desenvolvimento de um projeto reside no fato de que uma versão operacional do programa não estará disponível antes que o projeto esteja perto do final, por conta do estado de bloqueio que a natureza linear do ciclo de vida clássico produz.

Comentários:

Perfeito! O modelo cascata segue uma abordagem linear de desenvolvimento de software, onde cada etapa tem que ser concluída antes de a próxima etapa começar. Isso significa que, ao longo do desenvolvimento, nenhuma versão operacional do programa estará disponível até que o projeto esteja próximo do final.

Gabarito: Correto

98276239072 - FABIANO CARBONELL DA SILVA

7. (CESPE / BANRISUL – 2022) O modelo cascata é frequentemente inadequado para os casos de desenvolvimento de soluções de software em que há ritmo acelerado e volume intenso de mudanças de características, todavia ele pode servir como um modelo de processo útil em situações nas quais os requisitos são fixos.

Comentários:

Perfeito! O Modelo em Cascata é projetado para trabalhar com requisitos fixos. Ele não é ideal para projetos que exigem mudanças frequentes, pois ele não é flexível o suficiente para lidar com mudanças. Ele também não é ideal para projetos de grande volume ou de ritmo acelerado, pois o processo é muito linear e demorado. Em projetos com requisitos fixos, no entanto, o modelo cascata pode ser útil porque permite gerenciar e controlar o processo de forma eficiente.

Gabarito: Correto

8. (CESPE / BANRISUL – 2022) O modelo do ciclo de vida clássico sugere uma abordagem sequencial para desenvolvimento de software, começando com a especificação dos requisitos do cliente, avançando pelas fases de planejamento e modelagem, e culminando no suporte contínuo do software concluído.

Comentários:

Perfeito! O Modelo do Ciclo de Vida Clássico oferece uma abordagem estruturada para o desenvolvimento de software, a qual segue um conjunto de etapas comuns que devem ser cumpridas para garantir que o software desenvolvido atenda às necessidades do cliente. Estas etapas permitem que o desenvolvedor defina os requisitos específicos do cliente, crie um plano detalhado para o desenvolvimento do software e crie um produto final que atenda às necessidades do cliente. Além disso, as etapas do modelo do ciclo de vida clássico também oferecem suporte ao longo do ciclo de vida do software desenvolvido para garantir que o produto final atenda às expectativas do cliente.

Gabarito: Correto

9. (CESPE / CNPQ – 2024) O envolvimento do cliente, o uso do protótipo em produção parcial, o rápido planejamento e o fácil gerenciamento são vantagens da prototipação.

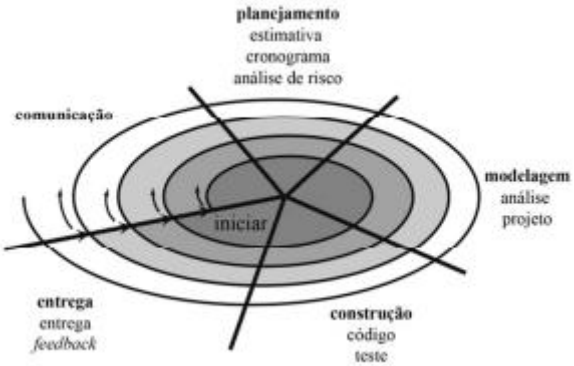
Comentários:

Normalmente, protótipos não são desenvolvidos com a intenção de serem usados em produção, mesmo que parcialmente. Eles são criados principalmente para validar ideias, conceitos e requisitos junto aos usuários ou stakeholders antes do desenvolvimento da solução final. Utilizar um protótipo em um ambiente de produção pode levar a problemas de estabilidade, segurança e manutenção, visto que protótipos frequentemente não são construídos com a robustez necessária para ambientes de produção.

Gabarito: Errado

98276239072 - FABIANO CARBONELL DA SILVA

10. (CESPE / MC – 2022) Conforme o modelo espiral da figura a seguir, o software será desenvolvido em uma série de versões evolucionárias, consistindo em um protótipo, nas primeiras iterações, e, progressivamente, em versões cada vez mais sofisticadas, nas iterações posteriores.



Comentários:

A figura ilustra o modelo espiral de desenvolvimento de software, que enfatiza a importância de avaliação e redução de riscos. Em cada ciclo espiral, novos elementos do projeto são planejados, analisados e desenvolvidos. Este modelo permite a criação de protótipos nas fases iniciais e refinamentos contínuos, assegurando que cada iteração do produto agregue valor e funcionalidades adicionais, ao mesmo tempo em que o feedback e o conhecimento do risco são melhorados progressivamente.

Gabarito: Correto

11. (CESPE / FUNPRESP-EXE - 2022) No modelo em espiral de desenvolvimento de software, cada giro ou loop da espiral representa uma fase do processo de software.

Comentários:

O modelo espiral é uma abordagem iterativa e evolutiva ao desenvolvimento de software, que combina elementos de design sequencial (modelo cascata) e prototipagem. Em cada ciclo da espiral, o projeto passa por quatro fases: planejamento, análise de riscos, engenharia e avaliação do cliente. Esta abordagem é particularmente útil para projetos grandes, complexos e de alto risco.

Gabarito: Correto

12. (CESPE / BANRISUL – 2022) Uma descrição ideal de um componente de software reutilizável deve ser feita com base no modelo 3C, que significa composição, conteúdo e contexto.

Comentários:

De acordo com Roger Pressman: "*Um componente de software reutilizável pode ser descrito de várias maneiras; porém, uma descrição ideal engloba aquilo que Tracz denominou modelo 3C –* ***conceito****,* ***conteúdo*** *e* ***contexto*** *–, uma descrição daquilo que o componente faz, como isso é*

98276239072 - FABIANO CARBONELL DA SILVA

obtido com conteúdo que pode ficar oculto para usuários eventuais e que precisa ser conhecido apenas por aqueles que pretendem modificar ou testar o componente e onde o componente reside em seu domínio de aplicabilidade".

Gabarito: Errado

13. (CESPE / DPE-RO – 2021) Um analista deve escolher uma metodologia de desenvolvimento para elaborar o planejamento do ciclo de vida de um produto de software de larga escala. O sistema é inédito e o reúso de código semelhante não deve ser considerado como base para o novo desenvolvimento. O analista deve considerar, ainda, a necessidade de reduzir os riscos em todas as fases do projeto, pois é provável que os requisitos sejam aprimorados e mudem ao longo do processo. Entre os riscos a serem mitigados, está o de não ter sido contratado pessoal de software suficiente para construir o produto, além de a equipe contratada não ter experiência suficiente no desenvolvimento de produtos em larga escala. Ainda, há o risco de o fornecedor do hardware necessário ao projeto não entregar todas as estações clientes no prazo do contrato.

Nessa situação hipotética, para a metodologia do processo de software em questão, é mais apropriado o uso do:

a) modelo codificar-e-corrigir.
b) modelo espiral.
c) modelo integração e configuração.
d) modelo em cascata.
e) modelo baseado em protótipos.

Comentários:

Observem que o analista busca reduzir o risco em todas as fases do projeto, logo podemos eliminar de cara o modelo em cascata, visto que ele atrasa a redução de riscos, fazendo-a somente nas fases finais. Também podemos eliminar o modelo corrigir-e-codificar (Code and Fix), pois nele é difícil avaliar a qualidade e os riscos do projeto. Esse é mal considerado uma metodologia, dado que ele envolve ir desenvolvendo e corrigindo erros por experimentação. O modelo integração e configuração é baseado em reúso, logo também deve ser eliminado. O modelo baseado em protótipos é utilizado quando o cliente não definiu detalhadamente os requisitos do software, logo não faz sentido aqui. Por fim, o modelo mais adequado para a redução dos riscos é o modelo espiral, que é orientado à redução de riscos.

Gabarito: Letra B

14. (CESPE / Polícia Federal – 2021) Embora não seja dirigido a riscos, o modelo de desenvolvimento de sistemas espiral de Boehm inclui, em seu framework, a etapa de análise e validação dos requisitos.

Comentários:

98276239072 - FABIANO CARBONELL DA SILVA

O Modelo em Espiral foi proposto originalmente, em 1988, por Boehm. Sua ideia era representar um processo de software orientado a riscos. Em vez de representar o processo como uma sequência de atividades com algum retorno entre uma atividade e outra, o processo é representado como uma espiral. Ele combina atividades de desenvolvimento com aspectos gerenciais (Planejamento, Tomada de Decisão, Análise de Riscos, etc).

Gabarito: Errado

15. (CESPE / SERPRO – 2021) No modelo formal, as etapas do desenvolvimento do software incluem especificação formal para definição de requisitos, refinamento para concepção de projeto e prova para a verificação.

Comentários:

As etapas são especificação formal para definição de requisitos, refinamento para concepção de projeto, síntese para implementação, prototipagem para a validação e prova para a verificação. Logo, todas essas listadas no enunciado estão contempladas.

Gabarito: Correto

98276239072 - FABIANO CARBONELL DA SILVA

Questionário de Revisão e Aperfeiçoamento

A ideia do questionário é elevar o nível da sua compreensão no assunto e, ao mesmo tempo, proporcionar uma outra forma de revisão de pontos importantes do conteúdo, a partir de perguntas que exigem respostas subjetivas.

São questões um pouco mais desafiadoras, porque a redação de seu enunciado não ajuda na sua resolução, como ocorre nas clássicas questões objetivas.

O objetivo é que você realize uma autoexplicação mental de alguns pontos do conteúdo, para consolidar melhor o que aprendeu ;)

Além disso, as questões objetivas, em regra, abordam pontos isolados de um dado assunto. Assim, ao resolver várias questões objetivas, o candidato acaba memorizando pontos isolados do conteúdo, mas muitas vezes acaba não entendendo como esses pontos se conectam.

Assim, no questionário, buscaremos trazer também situações que ajudem você a conectar melhor os diversos pontos do conteúdo, na medida do possível.

É importante frisar que não estamos adentrando em um nível de profundidade maior que o exigido na sua prova, mas apenas permitindo que você compreenda melhor o assunto de modo a facilitar a resolução de questões objetivas típicas de concursos, ok?

Nosso compromisso é proporcionar a você uma revisão de alto nível! Vamos ao nosso questionário:

Perguntas

1. **O que é Engenharia de Software segundo a IEEE?**
2. **Qual é o objetivo principal da Engenharia de Software?**
3. **O que define um ciclo de vida na Engenharia de Software?**
4. **Quais são as fases comuns em um ciclo de vida de software?**
5. **O que é um Modelo de Ciclo de Vida de Software?**
6. **O que caracteriza o Modelo em Cascata?**
7. **Quais são as vantagens do Modelo em Cascata?**
8. **Quais são as desvantagens do Modelo em Cascata?**
9. **O que é o Modelo Iterativo e Incremental?**

98276239072 - FABIANO CARBONELL DA SILVA

10. Como o Modelo Iterativo e Incremental reduz riscos?

11. Qual é a diferença entre o Modelo Iterativo e o Incremental?

12. Como a abordagem incremental é vantajosa para o cliente?

13. O que é o RAD (Rapid Application Development)?

14. Quando o RAD é mais adequado?

15. Quais são as vantagens do RAD?

16. Quais são as desvantagens do RAD?

17. O que é prototipação no desenvolvimento de software?

18. O que significa decomposição na Engenharia de Software?

19. Qual é a função da fase de testes no ciclo de vida de software?

20. O que é um processo de software?

21. O que caracteriza um Modelo de Processo de Software?

22. Quais são as atividades fundamentais de um processo de software segundo Sommerville?

23. O que são ferramentas CASE?

24. Qual a relação entre Engenharia de Software e Engenharia de Sistemas?

25. Como a formalidade ajuda na Engenharia de Software?

26. O que caracteriza os modelos evolucionários na Engenharia de Software?

27. Qual a principal diferença entre os modelos evolucionário e incremental?

28. O que é prototipagem no desenvolvimento de software?

29. Quais são os objetivos principais da prototipagem?

30. O que é a prototipagem "throwaway" ou descartável?

31. Como o modelo em espiral combina elementos de outros modelos?

32. Quais são os quatro setores do modelo em espiral proposto por Boehm?

33. Como o modelo em espiral minimiza riscos no desenvolvimento de software?

98276239072 - FABIANO CARBONELL DA SILVA

34. Quando o modelo em espiral é mais recomendado?

35. O que são métodos formais na Engenharia de Software?

36. Em quais situações os métodos formais são aplicados?

37. Qual é o maior desafio no uso de métodos formais?

38. O que caracteriza o modelo de desenvolvimento baseado em componentes?

39. Quais são as vantagens do modelo baseado em componentes?

40. O que são componentes COTS?

41. Como o modelo baseado em componentes facilita o reúso de software?

42. O que é a Programação Orientada a Aspectos (POA)?

43. Qual o principal benefício da Programação Orientada a Aspectos?

44. O que são interesses principais e interesses ortogonais na POA?

45. Como a POA complementa a Programação Orientada a Objetos (POO)?

46. O que são aspectos na POA?

47. Quando utilizar a prototipagem no processo de desenvolvimento de software?

48. O que é o desenvolvimento exploratório ou evolucionário?

49. O que é um protótipo funcional?

50. Qual o principal objetivo do modelo em espiral?

98276239072 - FABIANO CARBONELL DA SILVA

Perguntas com Respostas

1. O que é Engenharia de Software segundo a IEEE?

É a aplicação de uma abordagem sistemática, disciplinada e quantificável para o desenvolvimento, operação e manutenção de software.

2. Qual é o objetivo principal da Engenharia de Software?

Desenvolver sistemas de software que sejam econômicos, confiáveis e que funcionem em máquinas reais, utilizando princípios sólidos de engenharia.

3. O que define um ciclo de vida na Engenharia de Software?

O ciclo de vida de software se refere às fases pelas quais o software passa, desde a concepção até sua retirada de produção.

4. Quais são as fases comuns em um ciclo de vida de software?

As fases incluem: planejamento, análise de requisitos, projeto, implementação, testes, entrega, operação e manutenção.

5. O que é um Modelo de Ciclo de Vida de Software?

É a representação das fases do ciclo de vida do software e de como essas fases se relacionam, descrevendo o processo de desenvolvimento do software.

6. O que caracteriza o Modelo em Cascata?

É um modelo sequencial onde cada fase do desenvolvimento deve ser concluída antes que a próxima comece, com foco em fases bem definidas e documentadas.

7. Quais são as vantagens do Modelo em Cascata?

O modelo é fácil de gerenciar, documenta cada fase e é adequado para projetos com requisitos bem definidos desde o início.

8. Quais são as desvantagens do Modelo em Cascata?

A principal desvantagem é a dificuldade de incorporar mudanças nos requisitos durante o desenvolvimento, atrasando a identificação de erros e riscos.

9. O que é o Modelo Iterativo e Incremental?

É uma abordagem que desenvolve o software em pequenas partes (incrementos) e realiza várias repetições (iterações), permitindo a entrega de versões parciais e funcionais.

98276239072 - FABIANO CARBONELL DA SILVA

10. Como o Modelo Iterativo e Incremental reduz riscos?

Ao entregar partes do software em várias iterações, ele permite a identificação precoce de problemas e ajustes contínuos, minimizando os riscos.

11. Qual é a diferença entre o Modelo Iterativo e o Incremental?

No modelo incremental, o software é desenvolvido em partes separadas e integradas ao final. No iterativo, o software é refinado a cada ciclo até alcançar a versão final.

12. Como a abordagem incremental é vantajosa para o cliente?

O cliente pode avaliar o sistema desde os primeiros incrementos e fornecer feedback para melhorias, o que facilita ajustes durante o desenvolvimento.

13. O que é o RAD (Rapid Application Development)?

O RAD é um modelo de desenvolvimento rápido, iterativo e incremental que visa ciclos curtos de desenvolvimento, utilizando reutilização de componentes já existentes.

14. Quando o RAD é mais adequado?

O RAD é ideal quando os requisitos são bem definidos, o projeto é de pequeno a médio porte, e há necessidade de entrega rápida sem grandes riscos técnicos.

15. Quais são as vantagens do RAD?

Ele permite o desenvolvimento rápido de aplicações, reduz o tempo de codificação manual e facilita o envolvimento direto do usuário.

16. Quais são as desvantagens do RAD?

O RAD exige recursos humanos qualificados, um ambiente de desenvolvimento caro e pode comprometer a padronização entre módulos e componentes.

17. O que é prototipação no desenvolvimento de software?

É uma técnica onde um protótipo do sistema é desenvolvido para obter feedback precoce do cliente, permitindo ajustes antes da implementação completa.

18. O que significa decomposição na Engenharia de Software?

Decomposição é o princípio de dividir um problema em partes menores e mais gerenciáveis, permitindo uma resolução mais eficaz e especializada.

19. Qual é a função da fase de testes no ciclo de vida de software?

98276239072 - FABIANO CARBONELL DA SILVA

A fase de testes verifica se o software atende às especificações e se as funcionalidades implementadas funcionam corretamente, incluindo testes de unidade, integração e sistema.

20. O que é um processo de software?

É o conjunto de atividades, métodos, práticas e transformações que guiam as pessoas no desenvolvimento de software, desde a concepção até a manutenção.

21. O que caracteriza um Modelo de Processo de Software?

É uma descrição abstrata que define as atividades principais de um processo de software e a ordem em que essas atividades devem ser executadas.

22. Quais são as atividades fundamentais de um processo de software segundo Sommerville?

Especificação, desenvolvimento, validação e evolução são as atividades fundamentais que estão presentes em qualquer processo de software.

23. O que são ferramentas CASE?

Ferramentas CASE (Computer-Aided Software Engineering) são usadas para apoiar o desenvolvimento de software, automatizando atividades como modelagem e documentação.

24. Qual a relação entre Engenharia de Software e Engenharia de Sistemas?

A Engenharia de Software faz parte da Engenharia de Sistemas, que se preocupa com a integração de software, hardware e outros componentes de um sistema complexo.

25. Como a formalidade ajuda na Engenharia de Software?

A formalidade reduz inconsistências ao seguir passos bem definidos, garantindo que o processo de desenvolvimento seja sistemático e documentado adequadamente.

26. O que caracteriza os modelos evolucionários na Engenharia de Software?

São modelos iterativos que permitem desenvolver versões cada vez mais completas do software, evoluindo conforme as necessidades de negócio e produto mudam ao longo do tempo.

27. Qual a principal diferença entre os modelos evolucionário e incremental?

No modelo incremental, cada iteração apresenta uma funcionalidade operacional. No evolucionário, as primeiras iterações podem incluir apenas modelos em papel ou protótipos.

28. O que é prototipagem no desenvolvimento de software?

É uma abordagem iterativa usada para entender melhor os requisitos do sistema ao desenvolver versões preliminares do software para obter feedback antes da implementação completa.

98276239072 - FABIANO CARBONELL DA SILVA

29. Quais são os objetivos principais da prototipagem?

Identificar e refinar requisitos de software desconhecidos, fornecer uma visão clara para os usuários sobre o sistema proposto e ajudar no processo de tomada de decisões.

30. O que é a prototipagem "throwaway" ou descartável?

É um processo de prototipagem em que o protótipo é descartado após auxiliar no entendimento dos requisitos, sem ser utilizado como base para o desenvolvimento final.

31. Como o modelo em espiral combina elementos de outros modelos?

Ele integra características do modelo cascata e da prototipagem, aplicando ciclos iterativos onde o desenvolvimento e a análise de riscos são tratados de forma contínua.

32. Quais são os quatro setores do modelo em espiral proposto por Boehm?

Os setores são: determinar objetivos, alternativas e restrições; avaliar alternativas e resolver riscos; desenvolver e testar; e planejar as próximas fases do projeto.

33. Como o modelo em espiral minimiza riscos no desenvolvimento de software?

Ele permite identificar e resolver riscos em cada iteração, utilizando protótipos e análises contínuas para garantir que os riscos técnicos sejam mitigados antes de avançar.

34. Quando o modelo em espiral é mais recomendado?

É recomendado para projetos complexos e de grande escala, onde a identificação e mitigação de riscos são essenciais para o sucesso do desenvolvimento.

35. O que são métodos formais na Engenharia de Software?

São abordagens que utilizam representações matemáticas rigorosas para especificar, desenvolver e verificar software, garantindo maior precisão e menos ambiguidade nos requisitos.

36. Em quais situações os métodos formais são aplicados?

São aplicados em sistemas críticos que exigem alta robustez e confiabilidade, como sistemas de controle de tráfego aéreo, onde falhas podem causar danos graves.

37. Qual é o maior desafio no uso de métodos formais?

O principal desafio é o alto custo e o tempo necessário para implementação, além da necessidade de uma equipe qualificada com conhecimento avançado em matemática e lógica.

38. O que caracteriza o modelo de desenvolvimento baseado em componentes?

98276239072 - FABIANO CARBONELL DA SILVA

É uma abordagem que se concentra na reutilização de componentes de software previamente desenvolvidos para acelerar o processo e reduzir custos de desenvolvimento.

39. Quais são as vantagens do modelo baseado em componentes?

A principal vantagem é a redução do tempo e dos custos de desenvolvimento, permitindo entregas mais rápidas e de melhor qualidade, com foco na reutilização de componentes.

40. O que são componentes COTS?

COTS são "Commercial Off-The-Shelf" (produtos prontos), ou seja, componentes de software adquiridos externamente que podem ser integrados ao sistema em desenvolvimento.

41. Como o modelo baseado em componentes facilita o reúso de software?

Ao usar componentes prontos e testados, o modelo permite reutilizar módulos já existentes, reduzindo a necessidade de desenvolvimento desde o início e acelerando o projeto.

42. O que é a Programação Orientada a Aspectos (POA)?

É um paradigma que separa os interesses transversais (como segurança e persistência) das funcionalidades principais, facilitando a modularização e manutenção do software.

43. Qual o principal benefício da Programação Orientada a Aspectos?

Ela permite uma maior modularidade no código, separando interesses que se espalham por várias partes do sistema, tornando a manutenção e evolução mais simples.

44. O que são interesses principais e interesses ortogonais na POA?

Interesses principais são as funcionalidades centrais de um módulo, enquanto os ortogonais são requisitos transversais, como segurança ou registro de logs, que afetam várias partes do sistema.

45. Como a POA complementa a Programação Orientada a Objetos (POO)?

Ela não substitui a POO, mas a complementa, ajudando a resolver problemas que a POO sozinha não consegue, como a separação clara de interesses transversais no código.

46. O que são aspectos na POA?

Aspectos são elementos que encapsulam funcionalidades que afetam diversas partes de um sistema, como sincronização ou controle de acesso, e não podem ser facilmente modularizados por classes ou objetos.

47. Quando utilizar a prototipagem no processo de desenvolvimento de software?

98276239072 - FABIANO CARBONELL DA SILVA

A prototipagem é indicada quando os requisitos do cliente estão indefinidos ou mal compreendidos, permitindo explorar soluções e validar requisitos antes do desenvolvimento final.

48. O que é o desenvolvimento exploratório ou evolucionário?

É um processo no qual o software evolui conforme novas características são adicionadas, trabalhando de perto com o cliente para explorar e ajustar os requisitos ao longo do tempo.

49. O que é um protótipo funcional?

É uma versão inicial do sistema usada para validar requisitos, testar soluções de projeto e identificar possíveis falhas, com o objetivo de refinar e melhorar o desenvolvimento.

50. Qual o principal objetivo do modelo em espiral?

O modelo em espiral visa desenvolver software de forma iterativa e progressiva, reduzindo riscos através de ciclos que combinam análise de riscos, prototipagem e validação constante.

98276239072 - FABIANO CARBONELL DA SILVA

Lista de Questões Estratégicas

1. (CESPE / CAU-BR – 2024) Uma vantagem do ciclo de vida do software em cascata é que ele permite modificações nos requisitos por meio de atualização das fases anteriores.

2. (CESPE / CAU-BR – 2024) No modelo em cascata, o ciclo de vida do software torna o processo de desenvolvimento estruturado.

3. (CESPE / CAU-BR – 2024) Quando se utiliza um modelo incremental, frequentemente o primeiro incremento é um produto essencial; após esse primeiro, se libera uma série de incrementos, que oferecem, progressivamente, maior funcionalidade ao cliente à medida que cada incremento é entregue.

4. (CESPE / TCE-SC – 2022) Os modelos de processo de software em cascata e incremental possuem abordagens distintas. O primeiro considera as atividades fundamentais do processo, representando cada uma delas em fases distintas, tais como especificação de requisitos, implementação e teste; o segundo intercala as atividades de especificação, desenvolvimento e validação em uma série de versões do sistema, ao longo do seu ciclo de vida.

5. (CESPE / PETROBRAS – 2022) No modelo de desenvolvimento Waterfall, a partir da coleta de requisitos e da elaboração do projeto desenvolve-se uma implementação inicial, que é apresentada para a apreciação dos usuários; esse ciclo continua com a criação de várias versões, até que o sistema final seja desenvolvido por meio da execução das etapas de desenvolvimento e testes de forma intercalada.

6. (CESPE / BANRISUL – 2022) Um dos problemas da aplicação do modelo cascata no desenvolvimento de um projeto reside no fato de que uma versão operacional do programa não estará disponível antes que o projeto esteja perto do final, por conta do estado de bloqueio que a natureza linear do ciclo de vida clássico produz.

7. (CESPE / BANRISUL – 2022) O modelo cascata é frequentemente inadequado para os casos de desenvolvimento de soluções de software em que há ritmo acelerado e volume intenso de mudanças de características, todavia ele pode servir como um modelo de processo útil em situações nas quais os requisitos são fixos.

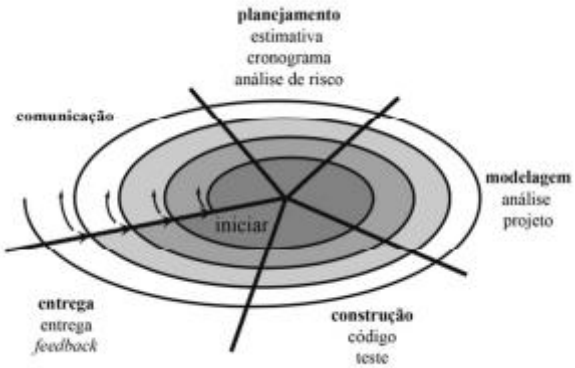
8. (CESPE / BANRISUL – 2022) O modelo do ciclo de vida clássico sugere uma abordagem sequencial para desenvolvimento de software, começando com a especificação dos requisitos do cliente, avançando pelas fases de planejamento e modelagem, e culminando no suporte contínuo do software concluído.

9. (CESPE / CNPQ – 2024) O envolvimento do cliente, o uso do protótipo em produção parcial, o rápido planejamento e o fácil gerenciamento são vantagens da prototipação.

10. (CESPE / MC – 2022) Conforme o modelo espiral da figura a seguir, o software será desenvolvido em uma série de versões evolucionárias, consistindo em um protótipo,

98276239072 - FABIANO CARBONELL DA SILVA

nas primeiras iterações, e, progressivamente, em versões cada vez mais sofisticadas, nas iterações posteriores.



11. (CESPE / FUNPRESP-EXE - 2022) No modelo em espiral de desenvolvimento de software, cada giro ou loop da espiral representa uma fase do processo de software.

12. (CESPE / BANRISUL – 2022) Uma descrição ideal de um componente de software reutilizável deve ser feita com base no modelo 3C, que significa composição, conteúdo e contexto.

13. (CESPE / DPE-RO – 2021) Um analista deve escolher uma metodologia de desenvolvimento para elaborar o planejamento do ciclo de vida de um produto de software de larga escala. O sistema é inédito e o reúso de código semelhante não deve ser considerado como base para o novo desenvolvimento. O analista deve considerar, ainda, a necessidade de reduzir os riscos em todas as fases do projeto, pois é provável que os requisitos sejam aprimorados e mudem ao longo do processo. Entre os riscos a serem mitigados, está o de não ter sido contratado pessoal de software suficiente para construir o produto, além de a equipe contratada não ter experiência suficiente no desenvolvimento de produtos em larga escala. Ainda, há o risco de o fornecedor do hardware necessário ao projeto não entregar todas as estações clientes no prazo do contrato.

Nessa situação hipotética, para a metodologia do processo de software em questão, é mais apropriado o uso do:

a) modelo codificar-e-corrigir.
b) modelo espiral.
c) modelo integração e configuração.
d) modelo em cascata.
e) modelo baseado em protótipos.

14. (CESPE / Polícia Federal – 2021) Embora não seja dirigido a riscos, o modelo de desenvolvimento de sistemas espiral de Boehm inclui, em seu framework, a etapa de análise e validação dos requisitos.

15. (CESPE / SERPRO – 2021) No modelo formal, as etapas do desenvolvimento do software incluem especificação formal para definição de requisitos, refinamento para concepção de projeto e prova para a verificação.

98276239072 - FABIANO CARBONELL DA SILVA

Referências Bibliográficas

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson Addison Wesley, 2019.

PRESSMAN, Roger S. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.

98276239072 - FABIANO CARBONELL DA SILVA

GABARITO

1. ERRADO
2. CORRETO
3. CORRETO
4. CORRETO
5. ERRADO
6. CORRETO
7. CORRETO
8. CORRETO
9. ERRADO
10. CORRETO
11. CORRETO
12. ERRADO
13. LETRA B
14. ERRADO
15. CORRETO

98276239072 - FABIANO CARBONELL DA SILVA

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.

Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.